

Deep Learning — End-to-End Cheat Sheet

A complete reference from fundamentals to deployment. Bookmark it. Use it daily.

Table of Contents

1. [The Big Picture](#)
2. [Math Foundations](#)
3. [Core Neural Network Concepts](#)
4. [Activation Functions](#)
5. [Loss Functions](#)
6. [Optimizers](#)
7. [Regularization](#)
8. [CNN — Convolutional Neural Networks](#)
9. [RNN / LSTM / GRU](#)
10. [Transformers & Attention](#)
11. [Generative Models](#)
12. [Training Pipeline](#)
13. [Hyperparameter Tuning](#)
14. [Data Preprocessing & Augmentation](#)
15. [Transfer Learning & Fine-Tuning](#)
16. [Model Evaluation Metrics](#)
17. [Frameworks & Tools](#)
18. [Deployment](#)
19. [Learning Roadmap](#)
20. [Key Papers to Read](#)
21. [Weight Initialization](#)
22. [Normalization Techniques — Deep Dive](#)
23. [Object Detection](#)
24. [Image Segmentation](#)
25. [Self-Supervised & Contrastive Learning](#)
26. [Graph Neural Networks](#)
27. [RLHF — Reinforcement Learning from Human Feedback](#)
28. [Efficient Training Techniques](#)
29. [LLMs — Large Language Models](#)

- 30. [Embeddings & Vector Databases](#)
- 31. [MLOps Pipeline](#)
- 32. [Debugging & Common Mistakes](#)
- 33. [Attention Mechanisms — Deep Dive](#)
- 34. [Common Deep Learning Datasets](#)
- 35. [Interview Cheat Sheet](#)
- 36. [Mathematics of Backpropagation](#)
- 37. [Recommended Learning Resources](#)

1. The Big Picture

Input Data → Preprocessing → Model → Loss → Optimizer → Trained Model → Deploy

Term	Meaning
Deep Learning	ML using neural networks with many layers
Neural Network	Layers of interconnected neurons (nodes)
Parameter	Learnable weight or bias in the network
Epoch	One full pass over the training dataset
Batch	Subset of data processed in one forward/backward pass
Inference	Using a trained model to make predictions

2. Math Foundations

Linear Algebra

Concept	Role in DL
Vector	Single data sample (feature vector)
Matrix	Batch of data or weight layers
Dot Product	Core operation in dense layers: $y = Wx + b$
Transpose	A^T — flip rows/cols

Concept	Role in DL
Eigenvalues	Used in PCA, optimization analysis

Calculus

Concept	Role in DL
Derivative	Rate of change — how loss changes w.r.t. weight
Chain Rule	Foundation of backpropagation
Gradient	Vector of partial derivatives
Jacobian	Matrix of all first-order partial derivatives

Probability & Stats

Concept	Role in DL
Probability Distribution	Modeling outputs (softmax → categorical dist.)
Bayes' Theorem	Probabilistic inference
MLE	Maximum Likelihood Estimation — training objective
KL Divergence	Measures difference between distributions (used in VAEs)
Cross-Entropy	Most common classification loss

3. Core Neural Network Concepts

Anatomy of a Neuron

Inputs ($x_1, x_2, \dots$) → [Weighted Sum: $z = \sum w_i x_i + b$] → [Activation $f(z)$] → Output

Forward Pass

Layer 1: $Z^1 = W^1 X + b^1, A^1 = f(Z^1)$

Layer 2: $Z^2 = W^2 A^1 + b^2, A^2 = f(Z^2)$

...

Output: $\hat{Y} = A^L$

Backpropagation

1. Compute loss: $L = \text{Loss}(\hat{Y}, Y)$

2. Compute $\partial L / \partial \hat{Y}$

3. Chain rule backwards through each layer

4. Update weights: $W = W - \alpha \cdot \partial L / \partial W$

Key Dimensions

Layer Type	Input Shape	Output Shape
Dense	(batch, in_features)	(batch, out_features)
Conv2D	(batch, H, W, C_in)	(batch, H', W', C_out)
LSTM	(batch, seq_len, features)	(batch, seq_len, hidden)

4. Activation Functions

Function	Formula	Range	Best Used For
ReLU	max(0, x)	[0, ∞)	Hidden layers (default choice)
Leaky ReLU	max(0.01x, x)	(-∞, ∞)	Fixing dead neurons
ELU	x if x>0 else α(e^x-1)	(-α, ∞)	Smoother than ReLU
GELU	x · Φ(x)	(-∞, ∞)	Transformers (BERT, GPT)
Sigmoid	1 / (1 + e^{-x})	(0, 1)	Binary output layer
Tanh	(e^x - e^{-x}) / (e^x + e^{-x})	(-1, 1)	RNN hidden states
Softmax	e^{x_i} / Σe^{x_j}	(0, 1), sums to 1	Multi-class output layer
Swish	x · sigmoid(x)	(-∞, ∞)	EfficientNet, modern CNNs

Rule of thumb: ReLU for hidden layers → Sigmoid/Softmax for output layer.

5. Loss Functions

Regression

Loss	Formula	Notes
MSE	$\frac{1}{n} \sum (\hat{y} - y)^2$	Penalizes large errors more
MAE	$\frac{1}{n} \sum \hat{y} - y $	
Huber	MSE if $ e \leq \delta$ MAE if $ e > \delta$	

Classification

Loss	Formula	Notes
Binary Cross-Entropy	$-\left[y \log \hat{y} + (1-y) \log(1-\hat{y})\right]$	Binary classification
Categorical Cross-Entropy	$-\sum y_i \log \hat{y}_i$	Multi-class (one-hot labels)
Sparse Cat. Cross-Entropy	Same but integer labels	More memory efficient
Focal Loss	$-\alpha_t(1-p_t)^\gamma \log(p_t)$	Handles class imbalance

Other

Loss	Use Case
KL Divergence	VAEs, probabilistic models
Contrastive Loss	Siamese networks
Triplet Loss	Face recognition, embeddings
SSIM Loss	Image reconstruction

6. Optimizers

Optimizer	Update Rule (simplified)	Notes
SGD	$W = W - \alpha \cdot \nabla W$	Simple, needs tuning
SGD + Momentum	$v = \beta v - \alpha \cdot \nabla W; W += v$	Faster convergence

Optimizer	Update Rule (simplified)	Notes
RMSProp	Divides lr by running avg of squared gradients	Good for RNNs
Adam	Combines momentum + RMSProp	Default choice for most tasks
AdamW	Adam + decoupled weight decay	Better generalization
Lion	Sign-based update	Memory efficient, newer
Adagrad	Adaptive lr per parameter	Sparse data tasks

Learning Rate Schedules

Schedule	Behavior
Step Decay	Reduce lr by factor every N epochs
Cosine Annealing	Smooth cosine curve decay
Warmup + Decay	Ramp up then decay (Transformers)
ReduceLROnPlateau	Reduce when val loss stagnates
Cyclical LR	Oscillate between min and max lr

7. Regularization

Technique	How It Works	When to Use
L1 (Lasso)	Adds $\lambda \Sigma$	w
L2 (Ridge)	Adds $\lambda \Sigma w^2$ to loss $\rightarrow$ small weights	General overfitting
Dropout	Randomly zero out p% neurons during training	Dense/Conv layers
Batch Norm	Normalize layer inputs per mini-batch	Deep networks
Layer Norm	Normalize across features per sample	Transformers, RNNs
Early Stopping	Stop when val loss stops improving	Always useful
Data Augmentation	Expand training set via transforms	Computer vision
Label Smoothing	Soften hard labels (e.g. $1 \rightarrow 0.9$)	Classification
Weight Decay	L2 applied to optimizer (AdamW)	Transformers

Technique	How It Works	When to Use
Gradient Clipping	Cap gradient norm to prevent explosion	RNNs, large models

8. CNN — Convolutional Neural Networks

Key Operations

Input Image → [Conv Layer] → [Activation] → [Pooling] → [Flatten] → [Dense] → Output

Convolution Parameters

Parameter	Description
Filters (K)	Number of feature detectors
Kernel Size	e.g. 3×3, 5×5 — spatial extent
Stride	Step size when sliding the kernel
Padding	<code>same</code> keeps size; <code>valid</code> reduces size
Dilation	Expand receptive field without more params

Output Size Formula

Output = $\lfloor (\text{Input} + 2P - K) / S \rfloor + 1$

Where: P = padding, K = kernel size, S = stride

Pooling Types

Type	Operation
Max Pooling	Take max value in window
Avg Pooling	Take average value in window
Global Avg Pool	Average entire feature map (replaces flatten)

Famous CNN Architectures

Model	Year	Key Innovation
LeNet-5	1998	Pioneer CNN
AlexNet	2012	Deep CNN + ReLU + Dropout
VGG-16/19	2014	Depth with 3×3 convs
GoogLeNet	2014	Inception modules
ResNet	2015	Skip connections (residuals)
DenseNet	2016	Dense skip connections
MobileNet	2017	Depthwise separable convs
EfficientNet	2019	Compound scaling
ConvNeXt	2022	CNN redesigned like ViT

9. RNN / LSTM / GRU

Vanilla RNN

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$
$$y_t = W_y h_t$$

⚠ Suffers from vanishing/exploding gradients on long sequences.

LSTM Gates

Gate	Formula	Purpose
Forget	$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	What to erase from cell
Input	$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	What new info to store
Cell	$\tilde{C} = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	Candidate values
Output	$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	What to output
Cell state	$C_t = f \odot C_{t-1} + i \odot \tilde{C}$	Long-term memory
Hidden	$h_t = o \odot \tanh(C_t)$	Short-term memory/output

GRU (Simplified LSTM)

$$z_t = \sigma(Wz \cdot [h_{t-1}, x_t]) \leftarrow \text{Update gate}$$
$$r_t = \sigma(Wr \cdot [h_{t-1}, x_t]) \leftarrow \text{Reset gate}$$
$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Fewer parameters than LSTM, often comparable performance.

Sequence Architectures

Type	Structure	Use Case
Many-to-One	Seq → Single output	Sentiment analysis
One-to-Many	Single → Seq	Image captioning
Many-to-Many	Seq → Seq (equal len)	POS tagging
Seq2Seq	Encoder-Decoder	Translation, summarization

10. Transformers & Attention

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

- Q = Query, K = Key, V = Value
- $\sqrt{d_k}$ prevents vanishing gradients with large dimensions

Multi-Head Attention

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

where $\text{head}_i = \text{Attention}(QW_iQ, KW_iK, VW_iV)$

Transformer Block

Input → [Multi-Head Attention] → Add & Norm
→ [Feed-Forward Network] → Add & Norm → Output

Positional Encoding

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/dmodel)})$$
$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/dmodel)})$$

Key Transformer Models

Model	Year	Type	Key Feature
Transformer	2017	Encoder-Decoder	"Attention is All You Need"
BERT	2018	Encoder only	Bidirectional, masked LM
GPT-2/3/4	2019+	Decoder only	Autoregressive generation
T5	2019	Enc-Dec	Text-to-text framework
ViT	2020	Encoder	Vision with patch embeddings
CLIP	2021	Dual encoder	Image-text contrastive learning
LLaMA	2023	Decoder only	Efficient open-source LLM

11. Generative Models

GAN (Generative Adversarial Network)

Generator (G): noise $z \rightarrow$ fake data

Discriminator (D): data $\rightarrow$ real/fake probability

Min-Max Game:

$$\min_G \max_D [E[\log D(x)] + E[\log(1 - D(G(z)))]]$$

Variant	Innovation
DCGAN	Conv layers in GAN
WGAN	Wasserstein distance, more stable
StyleGAN	High-quality face synthesis
CycleGAN	Unpaired image translation
Pix2Pix	Paired image-to-image translation

VAE (Variational Autoencoder)

Encoder: $x \rightarrow \mu, \sigma$ (latent space)

Reparameterization: $z = \mu + \sigma \cdot \epsilon, \epsilon \sim \mathcal{N}(0,1)$

Decoder: $z \rightarrow \hat{x}$

Loss = Reconstruction Loss + KL Divergence

Diffusion Models

Forward: $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_t$ (add noise gradually)

Reverse: $x_t \rightarrow \dots \rightarrow x_1 \rightarrow x_0$ (denoise with neural net)

Models: DDPM, DDIM, Stable Diffusion, DALL·E

12. Training Pipeline

Step-by-Step

1. Load & preprocess data
2. Split: train / val / test
3. Define model architecture
4. Choose loss function & optimizer
5. Training loop:
for epoch in epochs:
for batch in dataloader:
 $\hat{y} = \text{model}(x)$ # Forward pass
 $\text{loss} = \text{criterion}(\hat{y}, y)$ # Compute loss
 `optimizer.zero_grad()` # Clear gradients
 `loss.backward()` # Backprop
 `optimizer.step()` # Update weights
evaluate(val_data)
6. Evaluate on test set
7. Save model

PyTorch Quick Reference

python

```
import torch
import torch.nn as nn

# Define model
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 10)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)

model = Net()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

# Training step
model.train()
optimizer.zero_grad()
output = model(inputs)
loss = criterion(output, labels)
loss.backward()
optimizer.step()

# Inference
model.eval()
with torch.no_grad():
    predictions = model(test_inputs)
```

Keras / TensorFlow Quick Reference

```
python
```

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=20,
        batch_size=32, validation_split=0.2)
```

13. Hyperparameter Tuning

Key Hyperparameters

Hyperparameter	Typical Range	Notes
Learning Rate	1e-5 to 1e-1	Most impactful; try 3e-4 first
Batch Size	16 – 512	Larger = noisier but faster
Epochs	10 – 200	Use early stopping
Dropout Rate	0.1 – 0.5	0.3 is a good start
Hidden Units	64 – 4096	Depends on task complexity
Num Layers	2 – 100+	Start small, go deeper
Weight Decay	1e-5 to 1e-2	L2 regularization strength
Momentum	0.85 – 0.99	Usually 0.9

Tuning Strategies

Strategy	Description
Grid Search	Try all combinations (expensive)

Strategy	Description
Random Search	Random samples (often better)
Bayesian Optimization	Build surrogate model of performance
Population-Based	Evolve population of configs (PBT)
Manual Tuning	Informed intuition — fastest in practice

14. Data Preprocessing & Augmentation

Normalization

```
python

# Zero-mean, unit variance
x = (x - mean) / std

# Min-max scaling
x = (x - x_min) / (x_max - x_min)

# Per-channel ImageNet normalization
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]
```

Image Augmentation (torchvision)

```
python

from torchvision import transforms

transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ColorJitter(0.2, 0.2, 0.2),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

Text Preprocessing

```
Raw Text → Lowercase → Tokenize → Remove Stopwords
          → Stemming/Lemmatize → Encode (BPE/WordPiece)
```

Handling Imbalanced Data

Technique	Description
Oversampling	Duplicate minority samples (SMOTE)
Undersampling	Remove majority samples
Class Weights	Weight loss by inverse class freq
Focal Loss	Down-weight easy examples
Stratified Split	Preserve class ratios in splits

15. Transfer Learning & Fine-Tuning

Strategies

Strategy 1 — Feature Extractor:

Freeze all pretrained layers → Train only new head

Strategy 2 — Fine-Tuning (top layers):

Freeze early layers → Train last N layers + head

Strategy 3 — Full Fine-Tuning:

Unfreeze all layers → Train everything (small lr)

Strategy 4 — PEFT (Parameter-Efficient FT):

LoRA, Prefix Tuning, Adapters → Modify < 1% of params

PyTorch Example

```
python
```

```
import torchvision.models as models

model = models.resnet50(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace final layer
model.fc = nn.Linear(2048, num_classes)

# Only new layer is trained
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

LoRA (Low-Rank Adaptation)

$W' = W + \Delta W = W + A \cdot B$
where $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times k}$, rank $r \ll d$

Reduces trainable params from $d \times k$ to $r \times (d+k)$
Typically $r = 4, 8, \text{ or } 16$

16. Model Evaluation Metrics

Classification

Metric	Formula	Notes
Accuracy	Correct / Total	Misleading on imbalanced data
Precision	$TP / (TP + FP)$	Of predicted positives, how many correct
Recall	$TP / (TP + FN)$	Of actual positives, how many found
F1 Score	$2 \cdot P \cdot R / (P + R)$	Harmonic mean of P and R
AUC-ROC	Area under ROC curve	Threshold-independent
MCC	Balanced metric	Works for imbalanced classes

Regression

Metric	Formula	Notes
MSE	$\frac{1}{n} \sum (\hat{y} - y)^2$	Sensitive to outliers
RMSE	$\sqrt{\text{MSE}}$	Same units as target
MAE	$\frac{1}{n} \sum \hat{y} - y $	
R²	$1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$	1 = perfect, 0 = baseline
MAPE	$\frac{1}{n} \sum \frac{ y - \hat{y} }{ y }$	

Object Detection

Metric	Description
IoU	Intersection over Union of bounding boxes
mAP	Mean Average Precision across classes
COCO AP	AP at IoU 0.5:0.95

NLP

Metric	Use Case
BLEU	Machine translation
ROUGE	Summarization
Perplexity	Language models
BERTScore	Semantic similarity

17. Frameworks & Tools

Core Frameworks

Framework	Language	Best For
PyTorch	Python	Research, flexibility

Framework	Language	Best For
TensorFlow/Keras	Python	Production, deployment
JAX	Python	High-performance research
MXNet	Python	Scalable distributed training

Key Libraries

Library	Purpose
HuggingFace Transformers	Pretrained NLP/Vision models
timm	PyTorch image models
torchvision	CV transforms, datasets, models
torchtext	NLP utilities
OpenCV	Image processing
Albumentations	Fast image augmentation
PEFT	Parameter-efficient fine-tuning
bitsandbytes	8-bit quantization
DeepSpeed	Distributed training
ONNX	Cross-framework model export

Experiment Tracking

Tool	Key Feature
Weights & Biases	Rich dashboards, sweeps
MLflow	Open-source, all stages
TensorBoard	TensorFlow native, PyTorch supported
Neptune.ai	Collaboration focused
Comet ML	Auto-logging

18. Deployment

Export Formats

Format	Framework	Use Case
.pt / .pth	PyTorch	Save/load PyTorch model
TorchScript	PyTorch	Serialized, C++ deployment
ONNX	Cross-framework	Runtime optimization
TF SavedModel	TensorFlow	TF Serving, TFLite
TFLite	TensorFlow	Mobile/Edge devices
CoreML	Apple	iOS/macOS deployment
TensorRT	NVIDIA	GPU-optimized inference

Optimization Techniques

Technique	Description	Speedup
Quantization	Float32 → Int8/Float16	2–4×
Pruning	Remove near-zero weights	Variable
Knowledge Distillation	Train small model from large	Variable
Model Compilation	<code>torch.compile()</code>	1.5–2×
Batching Requests	Process multiple inputs at once	High throughput
Caching	Cache KV pairs (LLMs)	High latency reduction

Serving Options

- REST API

→ FastAPI + Uvicorn (simple, flexible)
- gRPC

→ TF Serving, Triton (low-latency)
- Serverless

→ AWS Lambda, GCP Cloud Functions
- Containers

→ Docker + Kubernetes
- Managed

→ AWS SageMaker, GCP Vertex AI, Azure ML
- Edge

→ TFLite, ONNX Runtime, CoreML

19. Learning Roadmap

● Beginner (0-3 months)

- Python + NumPy + Pandas
- Linear/Logistic Regression from scratch
- Neural networks: forward + backprop
- Build MLP on MNIST with PyTorch/Keras
- Understand loss, optimizer, training loop

● Intermediate (3-6 months)

- CNNs — build image classifier
- Transfer learning with ResNet/EfficientNet
- RNNs, LSTMs — sentiment analysis
- Data augmentation & regularization
- Experiment tracking (W&B or MLflow)
- Train on GPU (Google Colab / Kaggle)

● Advanced (6-12 months)

- Transformers from scratch
- Fine-tune BERT / GPT-2 on custom data
- Object detection (YOLO, Faster R-CNN)
- Segmentation (U-Net, Mask R-CNN)
- GANs and Diffusion Models
- Distributed training basics

● Expert (12+ months)

- Read & implement papers from scratch
- RLHF, Constitutional AI concepts
- LLM fine-tuning (LoRA, QLoRA)
- Custom CUDA kernels
- Full MLOps pipeline
- Research contributions / Kaggle medals

20. Key Papers to Read

Year	Paper	Concept
1986	Rumelhart et al.	Backpropagation
1989	LeCun et al.	Convolutional Networks

Year	Paper	Concept
1997	Hochreiter & Schmidhuber	LSTM
2012	Krizhevsky et al.	AlexNet — Deep Learning resurgence
2014	Goodfellow et al.	GANs
2015	He et al.	ResNet — skip connections
2017	Vaswani et al.	"Attention is All You Need"
2018	Devlin et al.	BERT
2020	Brown et al.	GPT-3 — few-shot learning
2020	He et al.	MoCo — contrastive learning
2021	Dosovitskiy et al.	ViT — Vision Transformer
2021	Radford et al.	CLIP
2022	Ho et al.	DDPM — Diffusion Models
2022	Hu et al.	LoRA
2023	Touvron et al.	LLaMA

 Quick Decision Guide

Task type?

- └─ Images/Video → CNN or ViT
- └─ Text/NLP → Transformer (BERT/GPT family)
- └─ Time Series → LSTM, GRU, or Temporal Conv
- └─ Generation → GAN, VAE, or Diffusion Model
- └─ Tabular Data → MLP, or try XGBoost first!
- └─ Multi-modal → CLIP-style dual encoders

Training issues?

- └─ Overfitting → Dropout, L2, more data, early stop
- └─ Underfitting → More layers, more epochs, less reg
- └─ Slow training → Larger lr, bigger batch, mixed precision
- └─ NaN loss → Clip gradients, reduce lr, check data
- └─ GPU OOM → Smaller batch, gradient checkpointing

Choosing an optimizer?

└─ Default: AdamW with cosine lr schedule + warmup

21. Weight Initialization

Poor initialization → vanishing/exploding gradients before training even begins.

Why It Matters

Too small weights → signals shrink to zero (vanishing)

Too large weights → signals explode (exploding)

Just right → unit variance through each layer

Initialization Methods

Method	Formula (std)	Best For
Zero Init	All zeros	✗ Never for weights (symmetry problem)
Random Normal	$N(0, 0.01)$	Small networks only
Xavier / Glorot	$\sqrt{2 / (\text{fan_in} + \text{fan_out})}$	Sigmoid, Tanh activations
He / Kaiming	$\sqrt{2 / \text{fan_in}}$	ReLU and variants
LeCun	$\sqrt{1 / \text{fan_in}}$	SELU activations
Orthogonal	Random orthogonal matrix	RNNs, deep residual nets

PyTorch Examples

```
python

import torch.nn as nn

# He initialization (default for Conv/Linear with ReLU)
nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')

# Xavier initialization
nn.init.xavier_uniform_(layer.weight)

# Bias always initialized to zero
nn.init.zeros_(layer.bias)
```

fan_in vs fan_out

fan_in = number of input units to the layer
fan_out = number of output units from the layer

For Conv2D: fan_in = kernel_h × kernel_w × C_in
fan_out = kernel_h × kernel_w × C_out

22. Normalization Techniques — Deep Dive

Method	Normalizes Over	Used In
Batch Norm	Batch dimension (N)	CNNs, MLPs
Layer Norm	Feature dimension (C)	Transformers, RNNs
Instance Norm	Spatial (H×W) per sample	Style transfer
Group Norm	Groups of channels	Object detection (small batch)
RMS Norm	RMS of features	LLaMA, modern LLMs
Spectral Norm	Weight matrix spectral radius	GANs (discriminator)

Batch Normalization — Full Detail

Input: $x \in \mathbb{R}^{B \times N}$ (B = batch, N = features)

1. Compute mean: $\mu_B = (1/B) \sum_i x_i$
2. Compute var: $\sigma^2_B = (1/B) \sum_i (x_i - \mu_B)^2$
3. Normalize: $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$
4. Scale & shift: $y_i = \gamma \hat{x}_i + \beta$ (γ, β learnable)

At inference: use running mean/var collected during training

Layer Normalization — Full Detail

Input: $x \in \mathbb{R}^{B \times N}$

Normalize over the N features for EACH sample independently:

$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)}$ per sample
 $y = \gamma \hat{x} + \beta$

No batch statistics → works for batch size = 1 → ideal for NLP

When to Place Norm Layers

Original ResNet: Conv → BN → ReLU → Conv → BN → + skip → ReLU
Pre-Activation: BN → ReLU → Conv → BN → ReLU → Conv → + skip ← often better
Transformer: Pre-LN (before attention) is more stable than Post-LN

23. Object Detection

Problem Definition

Input: Image ($H \times W \times 3$)
Output: List of (class_label, confidence, x1, y1, x2, y2) per object

Two-Stage Detectors

Model	Speed	Accuracy	Key Idea
R-CNN	Slow	Good	Region proposals + CNN per region
Fast R-CNN	Faster	Better	Single CNN + RoI Pooling
Faster R-CNN	Real-time	Best 2-stage	Region Proposal Network (RPN)
Mask R-CNN	Moderate	+ Segmentation	Adds mask head to Faster R-CNN

Faster R-CNN pipeline:
Image → Backbone (ResNet) → Feature Map → RPN → RoI Align → Classifier + BBox Regressor

One-Stage Detectors (YOLO Family)

Version	Year	Innovation
YOLOv1	2016	Single pass, grid-based detection
YOLOv3	2018	Multi-scale detection with FPN
YOLOv5	2020	PyTorch, easy to use, very fast
YOLOv8	2023	Anchor-free, best accuracy/speed trade-off

Version	Year	Innovation
RT-DETR	2023	Real-time DETR, no NMS needed

Key Concepts

Concept	Description
Anchor Boxes	Pre-defined bounding box shapes/scales
Anchor-Free	Predict box center + offsets directly
NMS	Non-Maximum Suppression — removes duplicate boxes
IoU	Intersection over Union: area of overlap / area of union
FPN	Feature Pyramid Network — multi-scale feature maps
RoI Align	Bilinear interpolation for precise feature extraction

Loss for Detection

Total Loss = $\lambda_{\text{cls}} \cdot L_{\text{classification}}$
 $+ \lambda_{\text{box}} \cdot L_{\text{bounding_box}}$ (SmoothL1 or CIoU)
 $+ \lambda_{\text{obj}} \cdot L_{\text{objectness}}$

24. Image Segmentation

Task Types

Task	Output	Example
Semantic Segmentation	Per-pixel class label	Road vs car vs sky
Instance Segmentation	Per-object mask	Car #1, Car #2, ...
Panoptic Segmentation	Semantic + Instance combined	Full scene understanding

U-Net Architecture

Encoder (Contracting):
Block1: Conv→BN→ReLU→Conv→BN→ReLU → MaxPool ↓
Block2: ...

Block3: ...
Bottleneck

Decoder (Expanding):
UpConv → Concat(skip connection) → Conv→BN→ReLU → ...
UpConv → Concat → Conv→BN→ReLU → ...
1×1 Conv → Output (num_classes channels)

Key: Skip connections preserve spatial detail

Other Segmentation Models

Model	Year	Key Feature
FCN	2015	First fully convolutional segmentation
SegNet	2015	Encoder-decoder with pooling indices
DeepLab v3+	2018	Atrous (dilated) convolutions, ASPP
Mask R-CNN	2017	Instance segmentation
Segment Anything (SAM)	2023	Zero-shot, prompt-based
SAM 2	2024	Video + image segmentation

Segmentation Loss Functions

Cross-Entropy Loss: Standard; works for balanced classes
Dice Loss: $2 \cdot |A \cap B| / (|A| + |B|)$; handles class imbalance
Focal Loss: Down-weights easy pixels
BCE + Dice combined: Most robust for medical imaging

25. Self-Supervised & Contrastive Learning

┃ Train without labels by creating pretext tasks from the data itself.

Core Idea

Learn representations by predicting:
- Masked patches (MAE, BERT)
- Rotations, colorization
- Whether two views come from the same image (contrastive)

Contrastive Learning Methods

Method	Year	Key Idea
SimCLR	2020	Maximize agreement of augmented views
MoCo v1/v2	2020	Momentum encoder + queue
BYOL	2020	No negative pairs needed
SimSiam	2021	No negatives, no momentum encoder
DINO	2021	Self-distillation with no labels (ViT)
MAE	2022	Mask 75% of patches, reconstruct
DINOv2	2023	Large-scale self-supervised ViT

NT-Xent Loss (SimCLR)

For a batch of N images $\rightarrow 2N$ augmented views

Positive pair: two views of same image

Negative pairs: all other $2(N-1)$ views

$$L(i,j) = -\log \left[\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)} \right]$$

$\text{sim}(u,v) = u^T v / (\|u\| \|v\|)$ (cosine similarity)

τ = temperature hyperparameter (typically 0.07–0.5)

Masked Autoencoder (MAE)

1. Divide image into 16×16 patches
2. Randomly mask 75% of patches
3. Encode visible patches with ViT
4. Decode all patches (masked tokens = learnable vectors)
5. Reconstruct pixel values of masked patches
6. Loss = MSE on masked patches only

26. Graph Neural Networks (GNNs)

When to Use GNNs

- Molecular property prediction (drug discovery)
- Social network analysis

- Knowledge graphs
- Recommendation systems
- Circuit design, traffic routing

Graph Basics

Graph $G = (V, E)$
 V = nodes (entities), E = edges (relationships)
Node features: $X \in \mathbb{R}^{(N \times d)}$
Adjacency matrix: $A \in \mathbb{R}^{(N \times N)}$

Message Passing Framework

For each node v :

1. Aggregate messages from neighbors N_v :
 $m_v = \text{AGGREGATE}(\{h_u : u \in N_v\})$
2. Update node representation:
 $h_{v'} = \text{UPDATE}(h_v, m_v)$

Repeat for K layers $\rightarrow$ K -hop neighborhood

GNN Architectures

Model	Aggregation	Notes
GCN	Mean of neighbors	Simple, most common
GraphSAGE	Mean / Max / LSTM	Inductive (unseen nodes)
GAT	Attention-weighted	Learns neighbor importance
GIN	Sum (most expressive)	Theoretically equivalent to WL test
MPNN	Flexible	Molecular property prediction

PyTorch Geometric Quick Reference

python

```
from torch_geometric.nn import GCNConv
import torch.nn.functional as F

class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(in_channels, 64)
        self.conv2 = GCNConv(64, num_classes)

    def forward(self, x, edge_index):
        x = F.relu(self.conv1(x, edge_index))
        x = F.dropout(x, training=self.training)
        return self.conv2(x, edge_index)
```

27. Reinforcement Learning from Human Feedback (RLHF)

┃ The technique behind ChatGPT, InstructGPT, Claude, and most instruction-following LLMs.

Three-Stage Pipeline

- Stage 1 — Supervised Fine-Tuning (SFT):
Pretrained LLM + human-written demonstrations → SFT Model
- Stage 2 — Reward Model Training:
Pairs of model outputs ranked by humans →
Train reward model to predict human preference score
- Stage 3 — PPO Fine-Tuning:
Use PPO (RL algorithm) to maximize reward model score
while staying close to SFT model (KL penalty)

Key Concepts

Term	Description
Reward Model	Binary classifier or scalar scorer trained on preference pairs
PPO	Proximal Policy Optimization — most used RL algo for RLHF
KL Divergence Penalty	Prevents model from drifting too far from SFT baseline
Constitutional AI	Rules-based RLHF alternative (Anthropic)
DPO	Direct Preference Optimization — no RL needed, simpler

Term	Description
RLAIF	RL from AI Feedback — use another model instead of humans

PPO Objective (Simplified)

$$L(\theta) = E[\min(r(\theta) \cdot \hat{A}, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A})] - \beta \cdot \text{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}]$$

$r(\theta) = \pi_{\theta}(a|s) / \pi_{\text{old}}(a|s)$ (probability ratio)
 $\hat{A}$ = advantage estimate
 β = KL penalty coefficient

DPO (Simpler Alternative to PPO)

Instead of training a separate reward model + RL:
 Directly optimize on preference data with:

$$L_{\text{DPO}} = -E[\log \sigma(\beta \cdot \log(\pi_{\theta}(y_w|x)/\pi_{\text{ref}}(y_w|x)) - \beta \cdot \log(\pi_{\theta}(y_l|x)/\pi_{\text{ref}}(y_l|x)))]$$

y_w = preferred response, y_l = rejected response

28. Efficient Training Techniques

Mixed Precision Training

```
python

# PyTorch AMP (Automatic Mixed Precision)
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for batch in dataloader:
    optimizer.zero_grad()
    with autocast():          # FP16 forward pass
        output = model(inputs)
        loss = criterion(output, labels)
    scaler.scale(loss).backward() # Scaled backward
    scaler.step(optimizer)
    scaler.update()
```

FP16 uses half the memory and is 2–8× faster on modern GPUs.

Gradient Accumulation

```
python

# Simulate large batch with small GPU memory
accumulation_steps = 4

for i, (inputs, labels) in enumerate(dataloader):
    output = model(inputs)
    loss = criterion(output, labels) / accumulation_steps
    loss.backward()

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Gradient Checkpointing

```
python

# Trade compute for memory: recompute activations during backward
from torch.utils.checkpoint import checkpoint

# In model forward pass:
x = checkpoint(self.expensive_layer, x)
# Reduces memory by  $\sim\sqrt{N}$  for N layers
```

DataLoader Optimization

```
python

DataLoader(
    dataset,
    batch_size=64,
    num_workers=4,    # Parallel data loading
    pin_memory=True,  # Faster CPU→GPU transfer
    prefetch_factor=2, # Prefetch batches
    persistent_workers=True # Keep workers alive
)
```

Distributed Training

Data Parallelism: Same model on each GPU, split batches

```
torch.nn.DataParallel (deprecated)
torch.nn.parallel.DistributedDataParallel (preferred)
```

Model Parallelism: Split model layers across GPUs

Used when model doesn't fit on one GPU

Pipeline Parallelism: Split model into stages, GPUs process different micro-batches

Used in GPT-3/4 scale training

ZeRO (DeepSpeed): Partition optimizer states, gradients, params

Stage 1: optimizer states (~4× memory reduction)

Stage 2: + gradients (~8× reduction)

Stage 3: + parameters (linear scaling with GPUs)

DDP Quick Reference

```
python

# Launch with: torchrun --nproc_per_node=4 train.py
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group("nccl")
local_rank = int(os.environ["LOCAL_RANK"])
model = model.to(local_rank)
model = DDP(model, device_ids=[local_rank])
```

29. LLMs — Large Language Models

Anatomy of a Decoder-Only LLM

```
Input tokens → Token Embedding + Positional Encoding
→ N × [
  RMSNorm → Multi-Head Self-Attention (causal mask)
  RMSNorm → Feed-Forward (SwiGLU activation)
]
→ RMSNorm → LM Head (linear → softmax over vocab)
```

Key LLM Techniques

Technique	Description
Causal Masking	Each token attends only to previous tokens
KV Cache	Cache K and V matrices for fast autoregressive generation
RoPE	Rotary Positional Embeddings — better length generalization

Technique	Description
ALiBi	Attention with linear biases — no positional embeddings
SwiGLU	FFN activation: $\text{SwiGLU}(x, W, V) = \text{swish}(xW) \odot xV$
GQA	Grouped Query Attention — fewer KV heads → faster inference
MQA	Multi-Query Attention — single KV head
Flash Attention	IO-aware exact attention — 2–4× faster, less memory
Sliding Window	Attend to only last W tokens (Mistral)
MoE	Mixture of Experts — sparse activation of expert FFNs

Tokenization

Method	Vocab Size	Used In
BPE	32k–100k	GPT family
WordPiece	30k	BERT
SentencePiece	32k–64k	T5, LLaMA
Unigram	Variable	SentencePiece variant
Byte-level BPE	50k	GPT-2/3/4

Token Sampling Strategies

python

```
# Greedy: always pick highest probability
token = argmax(logits)

# Temperature scaling: soften/sharpen distribution
probs = softmax(logits / temperature) # T<1=focused, T>1=creative

# Top-k sampling: only sample from top k tokens
top_k_probs = keep_top_k(probs, k=50)

# Top-p (nucleus) sampling: sample from smallest set summing to p
top_p_probs = keep_top_p(probs, p=0.9)

# Beam search: keep top-B sequences at each step
# Better for translation, worse for open-ended generation
```

Common LLM Families (2025)

Family	Creator	Params	Open?
GPT-4o	OpenAI	Unknown	No
Claude 3.5 / 4	Anthropic	Unknown	No
Gemini 2.0	Google DeepMind	Unknown	No
LLaMA 3.3	Meta	8B–405B	Yes
Mistral / Mixtral	Mistral AI	7B–8×22B	Yes
Qwen 2.5	Alibaba	0.5B–72B	Yes
Phi-4	Microsoft	14B	Yes
DeepSeek-V3/R1	DeepSeek	671B MoE	Yes
Gemma 3	Google	1B–27B	Yes

30. Embeddings & Vector Databases

What Are Embeddings?

Text/Image/Audio → Neural Encoder → Dense Vector ∈ ℝ^d

Similar items → similar vectors (close in embedding space)
Used for: search, recommendation, RAG, clustering

Embedding Types

Type	Model Examples	Dim
Word	Word2Vec, GloVe, FastText	50–300
Sentence	sentence-transformers, E5	384–1536
Code	CodeBERT, StarEncoder	768
Image	CLIP ViT, DINOv2	512–1024
Multi-modal	CLIP, ImageBind	512–1024

Similarity Metrics

```
python

import numpy as np

# Cosine similarity (most common for embeddings)
cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Euclidean distance
eucl = np.linalg.norm(a - b)

# Dot product (for normalized vectors = cosine similarity)
dot = np.dot(a, b)
```

Vector Databases

Database	Best For	Notes
Chroma	Local dev, RAG prototypes	In-memory or persistent
Pinecone	Production managed service	Fully managed
Weaviate	Multi-modal, hybrid search	Open-source
Qdrant	High performance, Rust-based	Open-source
FAISS	Research, offline indexing	Facebook, no server
pgvector	Already using PostgreSQL	SQL + vector

Database	Best For	Notes
Milvus	Large-scale production	Cloud-native

FAISS Indexing Types

IndexFlatL2	— Exact brute force (small datasets)
IndexIVFFlat	— Inverted file index (faster, approximate)
IndexHNSWFlat	— Graph-based (fast, high recall)
IndexIVFPQ	— Product Quantization (compressed, lossy)

RAG (Retrieval-Augmented Generation)

1. Chunk documents → embed each chunk → store in vector DB
2. User query → embed query
3. Find top-k similar chunks (ANN search)
4. Inject chunks into LLM context: "Answer using: {chunks}\n\nQuestion: {query}"
5. LLM generates grounded answer

31. MLOps Pipeline

Full Production ML Lifecycle

Data Collection → Data Validation → Feature Engineering → Model Training → Model Evaluation → Model Registry → Serving → Monitoring → Retraining

Data Versioning & Pipelines

Tool	Purpose
DVC	Data version control (like Git for data)
Great Expectations	Data validation & quality checks
Apache Airflow	Workflow orchestration (DAGs)
Prefect	Modern Airflow alternative
Kubeflow	Kubernetes-native ML pipelines
ZenML	Framework-agnostic ML pipelines
Apache Spark	Large-scale data processing

Model Registry

```
python

# MLflow example
import mlflow

with mlflow.start_run():
    mlflow.log_param("lr", 0.001)
    mlflow.log_metric("val_accuracy", 0.94)
    mlflow.pytorch.log_model(model, "model")

# Register best model
mlflow.register_model("runs:/run_id/model", "MyModel")

# Transition to production
client.transition_model_version_stage("MyModel", version=3, stage="Production")
```

CI/CD for ML (GitHub Actions)

```
yaml

# .github/workflows/ml_pipeline.yml
on: [push]
jobs:
  train-and-evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run training
        run: python train.py --config config.yaml
      - name: Evaluate model
        run: python evaluate.py --threshold 0.90
      - name: Deploy if passes
        if: success()
        run: ./deploy.sh
```

Model Monitoring

What to Monitor	Tools
Data Drift	Evidently, Alibi Detect, WhyLabs
Prediction Drift	Same tools
Latency / Throughput	Prometheus + Grafana
Error Rates	Datadog, New Relic

What to Monitor	Tools
Feature Statistics	Feast, Tecton

Infrastructure Stack

Development: Jupyter + local GPU

Experiment: Google Colab / Kaggle / Lambda Labs

Training: AWS p3/p4 instances, GCP A100s, RunPod

Orchestration: Kubernetes + Helm

Serving: FastAPI + Docker + NGINX

Scaling: HPA + Triton Inference Server

Monitoring: Prometheus + Grafana + Alertmanager

32. Debugging & Common Mistakes

Loss Not Decreasing

- ✓ Check: Is your learning rate too high or too low?
- ✓ Check: Are inputs normalized?
- ✓ Check: Is the data loading correctly? (visualize a batch)
- ✓ Check: Is the loss function correct for the task?
- ✓ Check: Are labels correct and in right format?
- ✓ Check: Try training on a tiny subset (5–10 samples) — should overfit

NaN / Inf Loss

- Cause 1: Exploding gradients → Clip gradients: `nn.utils.clip_grad_norm_(params, 1.0)`
- Cause 2: `log(0)` in cross-entropy → Use epsilon: `log(p + 1e-8)`
- Cause 3: Division by zero in normalization → Use epsilon in BN/LN
- Cause 4: Bad data (NaN in inputs) → Check: `assert not torch.isnan(x).any()`
- Cause 5: Learning rate too high → Reduce by $10\times$

Overfitting Checklist

1. Add Dropout (0.3–0.5 in FC layers, 0.1 in ViT)
2. Add L2 weight decay ($1e-4$ to $1e-2$)
3. More data or stronger augmentation
4. Reduce model capacity (fewer layers/units)
5. Early stopping
6. Label smoothing (0.1)
7. Use a pretrained model (less data needed)

Underfitting Checklist

- 1. Increase model capacity
- 2. Reduce regularization
- 3. Train for more epochs
- 4. Increase learning rate
- 5. Use a more expressive architecture
- 6. Check for bugs in data pipeline

GPU Memory Out of Error

- 1. Reduce batch size (halve it)
- 2. Use gradient checkpointing
- 3. Use mixed precision (FP16)
- 4. Reduce model size or sequence length
- 5. Use gradient accumulation to maintain effective batch size
- 6. Move embeddings to CPU if not training them

Common Bugs Table

Bug	Symptom	Fix
Forgot <code>zero_grad()</code>	Loss explodes after 1st epoch	Add <code>optimizer.zero_grad()</code> before <code>backward()</code>
Wrong loss for task	NaN or stuck at random	Match loss to output activation
Not calling <code>model.eval()</code>	Bad inference, dropout active	Call <code>model.eval()</code> and <code>torch.no_grad()</code>
Label mismatch	Accuracy stuck at 1/num_classes	Check label encoding (0-indexed, correct shape)
Data leakage	100% val accuracy suspiciously	Re-check train/val split
Wrong input normalization	Slow convergence	Normalize to dataset mean/std

33. Attention Mechanisms — Deep Dive

Types of Attention

Type	Description	Used In
Self-Attention	Query, Key, Value from same sequence	Transformers
Cross-Attention	Q from one seq, K/V from another	Encoder-Decoder

Type	Description	Used In
Causal (Masked)	Can't attend to future tokens	GPT, decoder
Multi-Head	Multiple attention heads in parallel	All Transformers
Sparse Attention	Attend to subset of positions	Longformer, BigBird
Flash Attention	IO-efficient exact attention	LLaMA, modern LLMs
Linear Attention	O(N) complexity approximation	Research

Self-Attention Step by Step

Input: $X \in \mathbb{R}^{(N \times d)}$ (N tokens, d dimensions)

1. Project to Q, K, V:
 $Q = X \cdot W_q, K = X \cdot W_k, V = X \cdot W_v$ (each $W_{q,k,v} \in \mathbb{R}^{(d \times d_k)}$)

2. Compute attention scores:
 $A = Q \cdot K^T / \sqrt{d_k}$ (scale by $\sqrt{d_k}$ to prevent vanishing gradient)

3. Apply causal mask (decoder only):
 $A[i,j] = -\infty$ if $j > i$ (future tokens masked)

4. Softmax:
 $A = \text{softmax}(A)$ ($A \in \mathbb{R}^{(N \times N)}$)

5. Weighted sum of values:
Output = $A \cdot V$ ($\mathbb{R}^{(N \times d_k)}$)

Multi-Head Attention Detail

h heads, each with $d_k = d_{\text{model}} / h$ dimensions

$\text{head}_i = \text{Attention}(Q \cdot W_{iq}, K \cdot W_{ik}, V \cdot W_{iv})$
 $\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^o$

Parameters per MHA layer:
 W_q, W_k, W_v : each $d \times d \rightarrow 3d^2$ params
 W^o : $d \times d \rightarrow d^2$ params
Total: $4d^2$

Complexity

Standard Self-Attention: $O(N^2 \cdot d)$ time, $O(N^2)$ memory

Flash Attention:	$O(N^2 \cdot d)$ time, $O(N)$ memory (tiling trick)
Sparse Attention:	$O(N \cdot \sqrt{N} \cdot d)$ time
Linear Attention:	$O(N \cdot d^2)$ time

Positional Encodings Comparison

Method	Learns?	Extrapolates?	Used In
Sinusoidal	No	Poorly	Original Transformer
Learned Absolute	Yes	No	BERT, ViT
Relative (T5)	Yes	Yes	T5, BERT variants
RoPE	No	Well	LLaMA, GPT-NeoX
ALiBi	No	Best	MPT, BLOOM
NoPE	—	N/A	Some linear attention

34. Common Deep Learning Datasets

Computer Vision

Dataset	Task	Size	Notes
MNIST	Digit classification	70K	Beginner benchmark
CIFAR-10/100	Image classification	60K	10/100 classes
ImageNet	Classification	1.2M	1000 classes, gold standard
COCO	Detection/Segmentation	330K	Most used detection benchmark
Pascal VOC	Detection	11K	20 classes
Open Images	Detection	9M	Google dataset
CelebA	Face attributes	200K	40 attributes
ADE20K	Segmentation	25K	150 semantic categories

NLP

Dataset	Task	Notes
SST-2	Sentiment	Movie reviews, binary
IMDB	Sentiment	50K movie reviews
SQuAD 1.1/2.0	QA	Reading comprehension
GLUE / SuperGLUE	Multi-task NLP benchmark	Standard NLP eval
WMT	Translation	Multiple language pairs
CommonCrawl	Pretraining data	Web text, massive
The Pile	LLM pretraining	825GB diverse text
HumanEval	Code generation	164 Python problems

Other

Dataset	Domain	Notes
LibriSpeech	Speech recognition	1000 hours audio
AudioSet	Audio classification	2M YouTube clips
ModelNet	3D shape recognition	40 categories
ShapeNet	3D models	51K objects
Kinetics-700	Video classification	700 action classes

35. Interview Cheat Sheet

Must-Know Conceptual Questions

Q: What is the vanishing gradient problem?

Gradients shrink exponentially as they propagate backward through many layers.
Especially bad with sigmoid/tanh. Solved by: ReLU, BatchNorm, ResNet skip connections, proper initialization (He/Xavier).

Q: Why does Batch Normalization help training?

1. Reduces internal covariate shift (keeps layer inputs roughly normalized)
2. Allows higher learning rates
3. Acts as mild regularizer (noise from batch statistics)
4. Makes network less sensitive to initialization

Q: What's the difference between L1 and L2 regularization?

L1: Promotes sparsity (many weights $\rightarrow$ exactly 0). Good for feature selection.
L2: Keeps all weights small (no zeros). More common, smoother optimization.
L1 gradient: constant $\pm\lambda$ (non-smooth at 0)
L2 gradient: linear $2\lambda w$ (smooth everywhere)

Q: Why use ReLU instead of sigmoid?

1. No vanishing gradient for positive inputs (gradient = 1, not <0.25)
2. Computationally faster (just $\max(0, x)$)
3. Promotes sparsity (many zeros in activations)

Downside: Dead neurons (negative inputs always output 0)
Fix: Leaky ReLU, ELU, GELU

Q: What is the difference between a Transformer encoder and decoder?

Encoder: Bidirectional self-attention (each token sees all tokens)
Used for: BERT, classification, encoding tasks

Decoder: Causal (masked) self-attention + cross-attention to encoder
Used for: GPT, generation, translation output

Encoder-Decoder: T5, BART — for seq2seq tasks

Q: What is gradient clipping and when do you use it?

Clips gradient norm to a max value before the update step:
if $\|\nabla\| > \text{max_norm}$: $\nabla = \nabla \times (\text{max_norm} / \|\nabla\|)$

Use for: RNNs, Transformers, any deep network prone to gradient explosion.
Typical value: 1.0

Q: Explain the Attention mechanism in one sentence.

Attention computes a weighted sum of values, where the weights measure how relevant each key is to the current query via dot product similarity.

Q: What is the difference between fine-tuning and transfer learning?

Transfer Learning: Use pretrained model as feature extractor (freeze weights)

Fine-Tuning: Update some or all pretrained weights on new task
(lower lr than training from scratch, typically 10–100×)

Q: What is knowledge distillation?

Train a small "student" model to mimic a large "teacher" model:

$\text{Loss} = \alpha \cdot \text{CE}(\text{student}, \text{labels}) + (1-\alpha) \cdot \text{KL}(\text{student_soft}, \text{teacher_soft})$

Soft labels (teacher's output probabilities) contain more info than hard labels.

Used to compress large models for edge deployment.

Q: How does Dropout work at test time?

During training: randomly zero out neurons with probability p

During inference: multiply all weights by (1-p) to maintain expected output
(or equivalently, no dropout but use keep_prob=1 in forward)

In PyTorch: `model.eval()` disables dropout automatically

36. Mathematics of Backpropagation

Chain Rule — The Engine of Backprop

For composition $f(g(x))$: $df/dx = (df/dg) \cdot (dg/dx)$

For neural network with L layers:

$$\partial L / \partial W^1 = \partial L / \partial A^L \cdot \partial A^L / \partial Z^L \cdot \partial Z^L / \partial A^{L-1} \cdot \dots \cdot \partial A^1 / \partial Z^1 \cdot \partial Z^1 / \partial W^1$$

Gradient of Common Operations

Operation	Forward	Gradient
Linear	$Z = WX + b$	$\partial L / \partial W = \partial L / \partial Z \cdot X^T$ $\partial L / \partial X = W^T \cdot \partial L / \partial Z$ $\partial L / \partial b = \text{sum}(\partial L / \partial Z, \text{axis=batch})$
ReLU	$A = \max(0, Z)$	$\partial L / \partial Z = \partial L / \partial A \cdot \mathbf{1}[Z > 0]$
Sigmoid	$A = \sigma(Z)$	$\partial L / \partial Z = \partial L / \partial A \cdot A(1-A)$
Tanh	$A = \tanh(Z)$	$\partial L / \partial Z = \partial L / \partial A \cdot (1 - A^2)$

Softmax+CE	Combined	$\partial L / \partial Z = A - Y$ (very clean!)
BatchNorm	Complex	See paper for full derivation

Backprop Through a Dense Layer

```
python

# Forward
Z = X @ W + b      # (batch, out)
A = relu(Z)        # (batch, out)

# Backward (given dL/dA from next layer)
dZ = dA * (Z > 0)   # ReLU gradient: 1 where Z>0, else 0
dW = X.T @ dZ       # (in, out)
db = dZ.sum(axis=0) # (out,)
dX = dZ @ W.T       # (batch, in) — passed to previous layer
```

37. Recommended Learning Resources

Books

Title	Author	Level
Deep Learning	Goodfellow, Bengio, Courville	Intermediate–Advanced
Hands-On ML with Scikit-Learn & TF	Aurélien Géron	Beginner–Intermediate
Programming PyTorch for DL	Ian Pointer	Beginner
The Little Book of DL	François Fleuret	Concise overview
Mathematics for ML	Deisenroth, Faisal, Ong	Math foundations
Natural Language Processing with Transformers	Tunstall et al.	NLP focus

Online Courses

Course	Provider	Best For
fast.ai Practical DL	fast.ai	Hands-on, top-down
CS231n	Stanford	Computer Vision
CS224n	Stanford	NLP

Course	Provider	Best For
CS285	UC Berkeley	Deep RL
Deep Learning Specialization	Coursera/deeplearning.ai	Structured beginner
Andrej Karpathy's YouTube	YouTube	Intuitive, excellent

Practice Platforms

Platform	What to Do
Kaggle	Competitions, notebooks, datasets
Papers With Code	SOTA benchmarks + code
Hugging Face	Models, datasets, spaces demos
Google Colab	Free GPU (T4), run experiments
Lightning AI	PyTorch Lightning + free GPUs

⚡ Updated Quick Decision Guide

- Task type?

 - └─ Images/Video → CNN (ResNet/EfficientNet) or ViT
 - └─ Text/NLP → Transformer (BERT/GPT/T5 family)
 - └─ Time Series → LSTM, GRU, or Temporal Conv Net
 - └─ Generation → Diffusion (images), GPT (text), GAN (fast)
 - └─ Tabular Data → Try XGBoost first! Then MLP / TabNet
 - └─ Multi-modal → CLIP-style dual encoders or LLaVA
 - └─ Graph Data → GCN / GAT / GraphSAGE
 - └─ Audio → Wav2Vec2, Whisper, 1D Conv
- Training issues?

 - └─ Loss NaN → Clip gradients, lower lr, check data
 - └─ Overfitting → Dropout, weight decay, more data, early stop
 - └─ Underfitting → Larger model, more epochs, lower regularization
 - └─ Slow convergence → Warmup lr, better init, BatchNorm
 - └─ GPU OOM → Smaller batch, fp16, gradient checkpointing
 - └─ Unstable GAN → Use WGAN-GP, spectral norm, smaller lr
- Deployment target?

 - └─ Server API → FastAPI + Docker + ONNX/TorchScript
 - └─ Mobile/Edge → TFLite / CoreML / ONNX Runtime

- └─ Browser → ONNX.js / TensorFlow.js
- └─ Real-time GPU → TensorRT on NVIDIA
- └─ Serverless → AWS Lambda + ONNX (watch cold starts)

Which optimizer?

- └─ Default: AdamW + cosine schedule + warmup
- Vision: SGD + momentum (0.9) + step decay (classical)
- LLMs: AdamW $\beta_1=0.9$ $\beta_2=0.95$, $lr=3e-4$, $min_lr=3e-5$
- GAN: Adam $\beta_1=0.5$ $\beta_2=0.999$, separate G/D learning rates

Last updated: 2025 | Based on current best practices in deep learning

💡 **Tip:** The best way to learn is to implement everything from scratch once, then use libraries after you understand what's happening under the hood.

🔑 **The 3 most important skills in DL:** (1) Debugging training runs, (2) Reading papers, (3) Fast experimentation. Master these and everything else follows.